Adversarial Search and Games

Consider the Connect-4 Game[1].

a) Give a complete problem formulation in the style of Chapter 3 (Section 3.1.1).

state: is a board of 6*7 specify which player moves first assign with '1' and the second will be '2' If no player in cell then I becomes '0'.

Initial state: Empty cell where all cell has value of 0.then player 1 move first
Goal state: if a player has four disc in a row (horizontally, vertically, or diagonally) connected.

Actions: For any state, list all the columns where a player can drop a disc (An action is valid if the chosen column is not full).

Action Costs: In games, especially zero-sum games like Connect-4, action costs are often uniform. Since both players alternate turns, and each move advances the game by one step, assigning a uniform cost of 1 per action is acceptable.

Transition Model:

When a player drops a disc into a column**.:**

- The disc occupies the lowest available cell in that column.
- state updates to reflect the new board configuration.
- turn switches to the other player.

b) Design and explain a heuristic function that, given the state of a Connect-4 game board, determines the quality of the state.

Line Evaluation:

[+] Connect-4 is all about creating a line of four discs horizontally, vertically, or diagonally.
[+] The heuristic should evaluate all possible lines on the board for both players.

Position Weighting:

[+] Central columns are more valuable because they offer more opportunities for creating lines.
[+] The heuristic should favor placing discs in central positions.

NOTE: Objective of the Heuristic Function is to estimate how close the player is to winning and recognize and account for the opponent's potential to win

c) Design and implement an algorithm (i.e., ***Agent***) that, given the state of a Connect-4 game board, determines the optimal move for the agent to maximize its chances of winning and block the opponent from winning.

i. ***AgentA***. Assume the opponent will pick their best move with probability, *p*, equals to 1 (*minmax algorithm*).

ii. ***AgentB***. Assume the opponent will pick their best move with probability, *p*, less than 1 (*expectimax algorithm*).

iii. For both agents, ***specify*** and ***justify*** the depth your algorithm will explore.

iv. For both agents, apply ***alpha-beta pruning*** whenever applicable.

d) Design and implement ***AgentC***, that just uses random moves (random algorithm). e) Play the following sets of games:

i. ***AgentA*** against ***AgentA*** (ten times)

ii. ***AgentA*** against ***AgentB***, *p* = *.5* (ten times)

iii. ***AgentA*** against ***AgentC*** (ten times)

iv. ***AgentB***, *p* = *.75* against ***AgentC*** (ten times)

v. ***AgentB***, *p* = *.5* against ***AgentC*** (ten times)

vi. ***AgentB***, *p* = *.25* against ***AgentC*** (ten times)

f) For each set of games, report the following:

i. The *win-to-loose ratio*, i.e., number of *wins* divided by the numbers of *losses*.

ii. The *min*, *max*, *average*, *standard deviation* of the length of the game. iii. The *min*, *max*, *average*, *standard deviation* of the number of pruning: alphas and beta.

iv. Comment on the results and your observations.

g) Comment on the overall results and your observations.

```
Simulating AgentA vs AgentA with p = 1.0
Results between AgentA and AgentA:
Wins: {'AgentA': 9, 'Draw': 1}
Win/Loss Ratio (AgentA to AgentA): 1.0
Game Lengths - Min: 12, Max: 43, Avg: 25.10, Std Dev: 9.22
Alpha Prunes - Min: 0, Max: 0, Avg: 0.00, Std Dev: 0.00
Beta Prunes - Min: 0, Max: 269, Avg: 92.47, Std Dev: 59.11

Simulating AgentA vs AgentB with p = 0.5
Results between AgentA and AgentB:
Wins: {'AgentA': 8, 'AgentB': 0, 'Draw': 2}
Win/Loss Ratio (AgentA to AgentB): inf
Game Lengths - Min: 21, Max: 43, Avg: 34.00, Std Dev: 7.00
Alpha Prunes - Min: 0, Max: 0, Avg: 0.00, Std Dev: 0.00
Beta Prunes - Min: 0, Max: 206, Avg: 39.31, Std Dev: 53.11

Simulating AgentA vs AgentC with p = 1.0
Results between AgentA and AgentC:
Wins: {'AgentA': 10, 'AgentC': 0, 'Draw': 0}
Win/Loss Ratio (AgentA to AgentC): inf
Game Lengths - Min: 7, Max: 27, Avg: 13.60, Std Dev: 6.20
Alpha Prunes - Min: 0, Max: 0, Avg: 0.00, Std Dev: 0.00
Beta Prunes - Min: 29, Max: 275, Avg: 99.84, Std Dev: 54.27

Simulating AgentB vs AgentC with p = 0.75
Results between AgentB and AgentC:
Wins: {'AgentB': 7, 'AgentC': 2, 'Draw': 1}
Win/Loss Ratio (AgentB to AgentC): 3.5
Game Lengths - Min: 33, Max: 43, Avg: 38.80, Std Dev: 2.71
```

Metric	AgentA vs AgentA	AgentA vs AgentB (p=0.5)	AgentA vs AgentC	AgentB (p=0.75) vs AgentC	AgentB (p=0.5) vs AgentC	AgentB (p=0.25) vs AgentC
Agent1 Wins	4	10	10	10	10	10
Agent2 Wins	4	0	0	0	0	0
Draws	2	0	0	0	0	0
Win-to-Loss Ratio	1.00					
Minimum Moves	38	35	7	4	4	4
Maximum Moves	42	35	19	9	9	9
Average Moves	40.70	35.00	9.20	6.30	6.90	6.20
Standard Deviation of Moves	1.57	0.00	4.05	1.70	2.33	1.55
Alpha Prunes Minimum	228	1,085	158	6	6	6
Alpha Prunes Maximum	262	1,085	872	7	6	7
Alpha Prunes Average	247.70	1,085.00	307.90	6.40	6.00	6.20
Alpha Prunes	11.48	0.00	241.70	0.52	0.00	0.42

Std. Deviation						
Beta Prunes Minimum	245	248	19	14	13	18
Beta Prunes Maximum	278	248	112	36	41	36
Beta Prunes Average	262.20	248.00	42.40	26.30	28.80	26.80
Beta Prunes Std. Deviation	10.83	0.00	36.14	7.26	9.11	5.73

AgentA vs. AgentA:

- **Evenly matched**, resulting in an equal number of wins and some draws.
- **High average game length** due to both agents playing optimally and blocking each other's winning strategies.
- **Significant pruning**, indicating complex decision trees that benefit from alpha-beta pruning.

AgentA vs. AgentB (p = 0.5):

- **AgentA won all games**, as AgentB does not always play optimally due to the 0.5 probability of making the best move.
- **Consistent game length** and **high alpha prunes**, showing AgentA's efficiency in exploring optimal moves while pruning suboptimal branches.

AgentA vs. AgentC:

- **AgentA dominated**, winning all games against the random strategy of AgentC.
- **Shorter game lengths** due to AgentC's unpredictable moves, which AgentA exploited.
- **Variable pruning counts**, reflecting the inconsistent nature of AgentC's gameplay.

AgentB vs. AgentC (varying p values):

- **AgentB consistently won** all games against AgentC across different probabilities (p = 0.75, 0.5, 0.25).
- **Game lengths decreased** slightly as the probability decreased, suggesting a more aggressive strategy when AgentB assumes the opponent is less likely to play optimally.
- **Pruning counts were lower** compared to AgentA, especially in alpha prunes, due to the probabilistic nature of the expect-max algorithm.

Observations

minmax Agents (AgentA):

- **Performance:**
  - Excel in both offensive and defensive play, consistently winning against less optimal opponents.
  - **Alpha-beta pruning** significantly enhances their efficiency by reducing the search space.
- **Game Characteristics:**
  - Games tend to be longer when playing against equally optimal agents due to mutual blocking strategies.
  - Against random agents, games are shorter as the minimax agent quickly capitalizes on mistakes.

Expectimax Agents (AgentB):

- **Performance:**
  - Perform well against random opponents like AgentC.
  - May struggle against fully optimal opponents if the probability model does not accurately reflect the opponent's behavior.
- **Pruning Efficiency:**
  - **Alpha-beta pruning** is less effective in expectimax due to the algorithm's consideration of probabilities over discrete optimal moves.
  - Lower pruning counts indicate less opportunity for pruning in the expectimax search tree.

Random Agents (AgentC):

Role and Performance:

Serve as a baseline for evaluating strategic agents.

Consistently outperformed due to the lack of strategic planning and foresight.

Games are generally shorter when playing against strategic agents, as their mistakes are quickly exploited.

Notes:

- This is an *individual-type* assignment, no group work is accepted.
- Submit the Report and the Code (as Appendix) in PDF.
- Source code also be submitted in a separate file(s). - No Zip Files.

[1] https://en.wikipedia.org/wiki/Connect_Four#:~:text=Connect%20Four%20(also%20known%20as,seven%2Dcolumn%20vertically%20suspended%20grid.